Employee Management System (EMS)

Software Analysis & Design Document

Student Name: _____	Group No: _____
Class No: _____	Date: 13/10/2026

Note: the task outline does not name a system, so this document applies every step of the outline to one example system, an **Employee Management System** (the outline's own example uses *Emp*). The same steps can be repeated for any other system.

1) SRS (Software Requirements Specification)

1.1 Purpose and Scope

EMS is a web application that lets a company manage its employees, departments, positions, daily attendance, leave requests and monthly payroll in one place, replacing paper files and spreadsheets.

1.2 Actors

- **Admin:** full control of all data and settings.
- **HR Officer:** manages employees, departments, leaves and payroll.
- **Employee:** views own profile, records attendance, requests leave, views payslips.

1.3 Functional Requirements

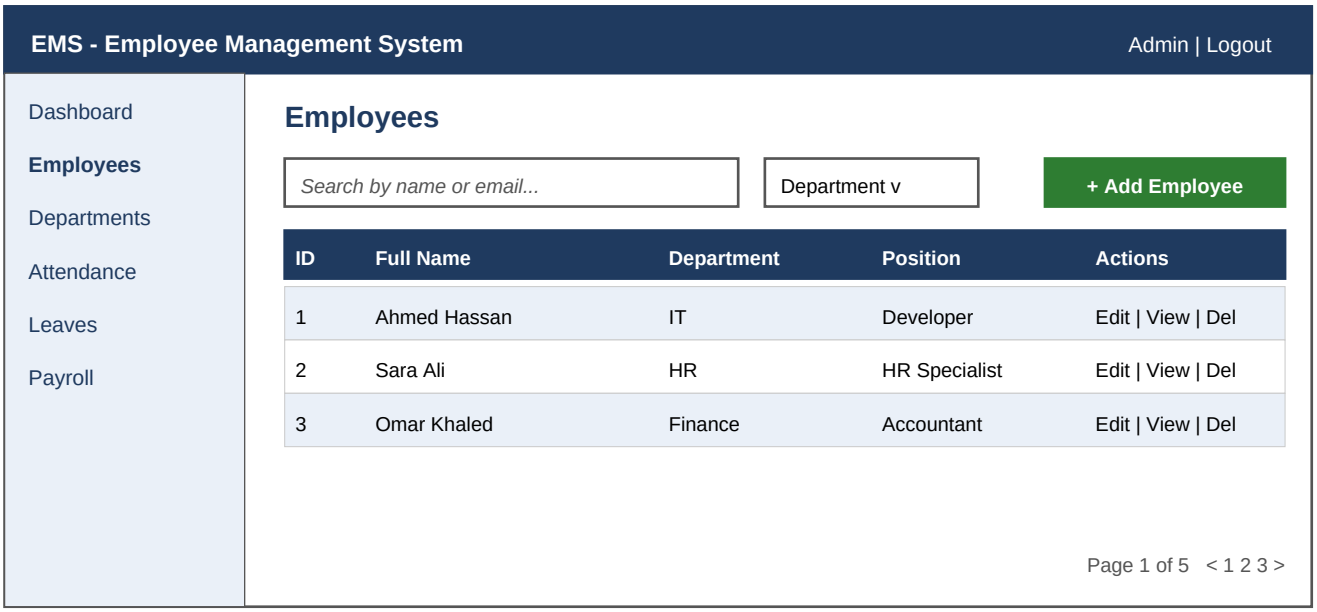
ID	Requirement	Actor
FR-01	The system shall authenticate users with email and password and apply role-based access.	All
FR-02	The system shall allow adding, editing, viewing, searching and deactivating employees.	Admin, HR
FR-03	The system shall manage departments and positions and assign employees to them.	Admin, HR
FR-04	The system shall record daily check-in and check-out and calculate worked hours.	Employee
FR-05	The system shall let employees submit leave requests and let HR approve or reject them.	Employee, HR
FR-06	The system shall calculate monthly payroll (base salary + bonus - deductions) and show payslips.	HR, Employee
FR-07	The system shall produce reports (headcount per department, attendance summary, leave balance).	Admin, HR

1.4 Non-Functional Requirements

- **Security:** hashed passwords, HTTPS, role-based authorization, protection against SQL injection and CSRF.
- **Performance:** pages load in under 3 seconds with up to 1,000 employees.
- **Usability:** responsive interface usable on desktop and mobile.
- **Maintainability:** layered code so that features can change without breaking other parts.
- **Availability:** 99% uptime during working hours; daily database backup.

2) UI (Design)

The interface uses a fixed top bar, a side menu and a content area. Below is the wireframe of the main screen, followed by the list of screens.



Screen	Main elements
Login	Email, password, Login button, error message.
Dashboard	Cards: total employees, present today, pending leaves; chart of employees per department.
Employees (list)	Search box, department filter, table, Add / Edit / Details / Delete buttons, paging.
Employee form	Full name, email, phone, hire date, department, position, salary, Save / Cancel.
Attendance	Check-in / Check-out buttons, today's record, monthly attendance table.
Leave requests	Request form (dates, reason); HR table with Approve / Reject.
Payroll	Month selector, payroll table, payslip details.

3) Classes, Calling and Business/System Classes Calling

The **business classes** (Employee, Department, ...) hold data and business rules. The **system classes** (controllers, services, repositories) organize how a request reaches the business classes and the database. The table shows who calls whom.

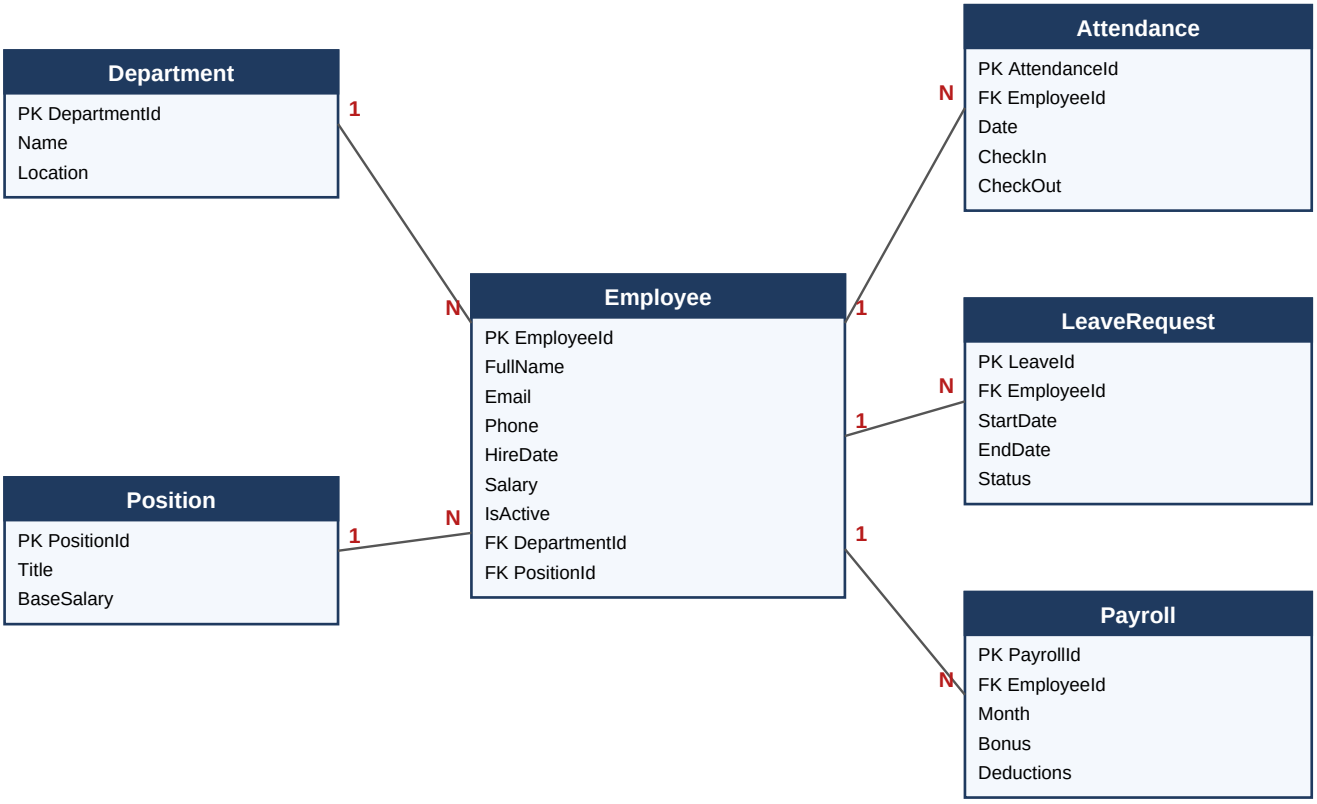
Caller	Calls	Purpose
EmployeesController	IEmployeeService (EmployeeService)	Handle the HTTP request and pass data to the business logic.
EmployeeService	IEmployeeRepository, Employee entity	Apply rules (e.g. unique email), create/update the Employee object.
EmployeeRepository	AppDbContext	Read and write Employee data through Entity Framework Core.
LeaveService	ILeaveRepository, LeaveRequest.Approve()	Validate dates and change the leave status.
PayrollService	IEmployeeRepository, Payroll.CalculateNetSalary()	Compute monthly payroll for each active employee.

Calling sequence for Add Employee:

```
User -> EmployeesController.Create(vm)
        -> EmployeeService.AddEmployee(dto)
                -> new Employee(...)          // business rules in the entity
                -> EmployeeRepository.Add(emp)
                        -> AppDbContext.SaveChanges() -> Database
        <- returns result -> Controller redirects to Employees list
```

4) Data Modelling: ERD

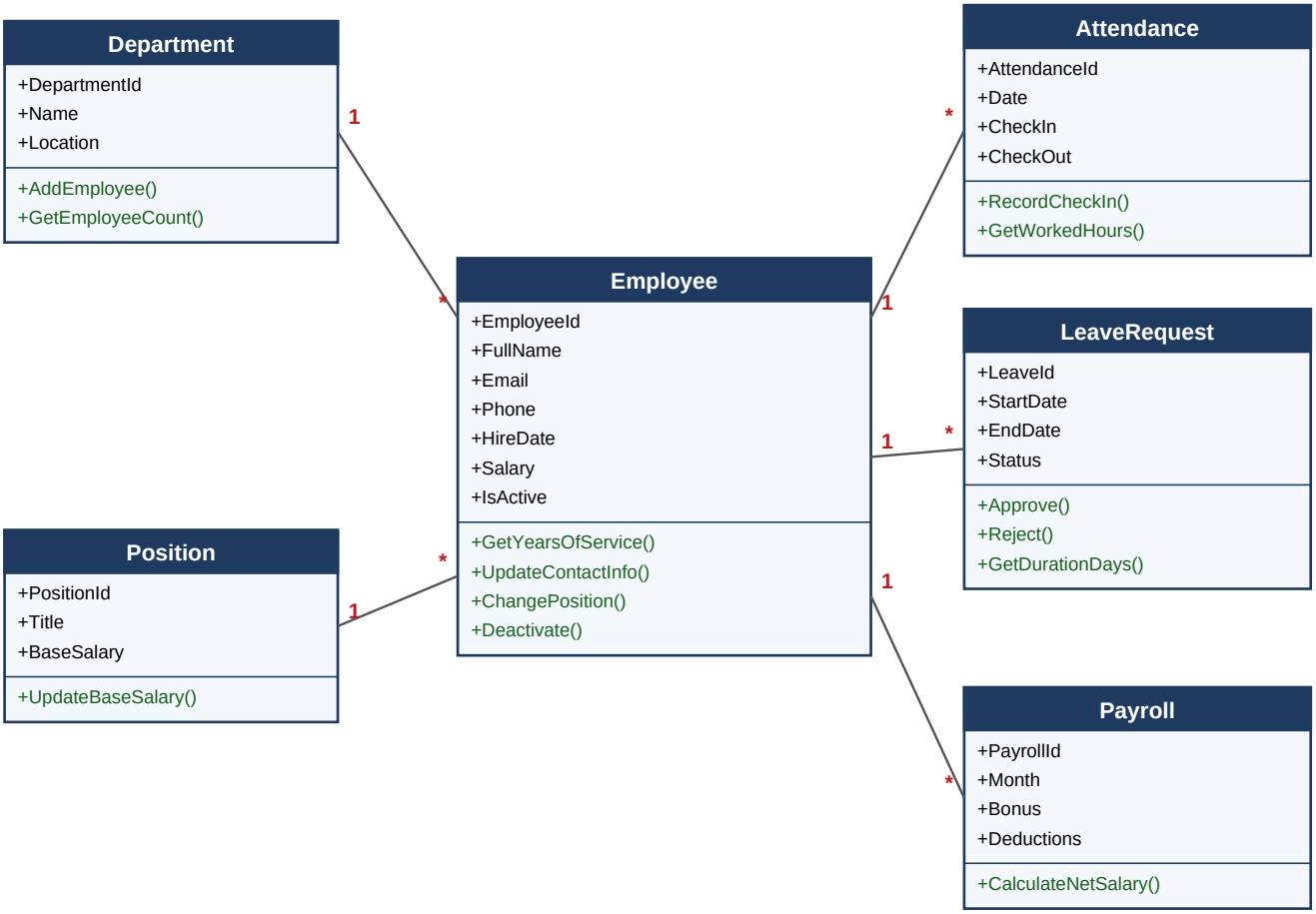
Each department and each position has many employees (one-to-many). Each employee has many attendance records, leave requests and payroll records (one-to-many). PK = primary key, FK = foreign key.



Entity	Key columns and notes
Department	DepartmentId (PK), Name (unique), Location.
Position	PositionId (PK), Title, BaseSalary.
Employee	EmployeeId (PK), FullName, Email (unique), Phone, HireDate, Salary, IsActive, DepartmentId (FK), PositionId (FK).
Attendance	AttendanceId (PK), EmployeeId (FK), Date, CheckIn, CheckOut. One record per employee per day.
LeaveRequest	LeaveId (PK), EmployeeId (FK), StartDate, EndDate, Status (Pending / Approved / Rejected).
Payroll	PayrollId (PK), EmployeeId (FK), Month, Bonus, Deductions. Net salary is calculated.

5) Class Diagram / Classes Specs Document (Req) - Shipping Quickly

The class diagram turns the ERD into object-oriented classes. Each class lists its **properties**, **behaviors** (methods) and **relations**. Having this specs document ready lets the team start coding at once and ship quickly.



Class Specs Document

Example: Emp (Employee) - Property / Behavior / Relation as in the outline.

Class	Properties	Behaviors	Relations
Employee	EmployeeId, FullName, Email, Phone, HireDate, Salary, IsActive	GetYearsOfService(), UpdateContactInfo(), ChangePosition(), Deactivate()	Belongs to 1 Department and 1 Position; has many Attendance, LeaveRequest, Payroll.
Department	DepartmentId, Name, Location	AddEmployee(), GetEmployeeCount()	Has many Employee.
Position	PositionId, Title, BaseSalary	UpdateBaseSalary()	Has many Employee.
Attendance	AttendanceId, Date, CheckIn, CheckOut	RecordCheckIn(), GetWorkedHours()	Belongs to 1 Employee.
LeaveRequest	LeaveId, StartDate, EndDate, Status	Approve(), Reject(), GetDurationDays()	Belongs to 1 Employee.
Payroll	PayrollId, Month, Bonus, Deductions	CalculateNetSalary()	Belongs to 1 Employee.

6) Architecture Pattern

6.1 Step 1 - Criteria

Before choosing a pattern, the system is evaluated against these criteria:

Criterion	EMS situation
Complexity of business rules	Medium: leave approval, payroll calculation, validations.
Maintainability and changes	High: rules and screens will change over time.
Testability	High: business logic should be unit-testable without a database or UI.
Team size and skills	Small student team that knows C# and ASP.NET MVC.
Technology independence	Should be possible to change the database or UI later.
Scalability / future growth	Moderate: more modules (recruitment, training) may be added.

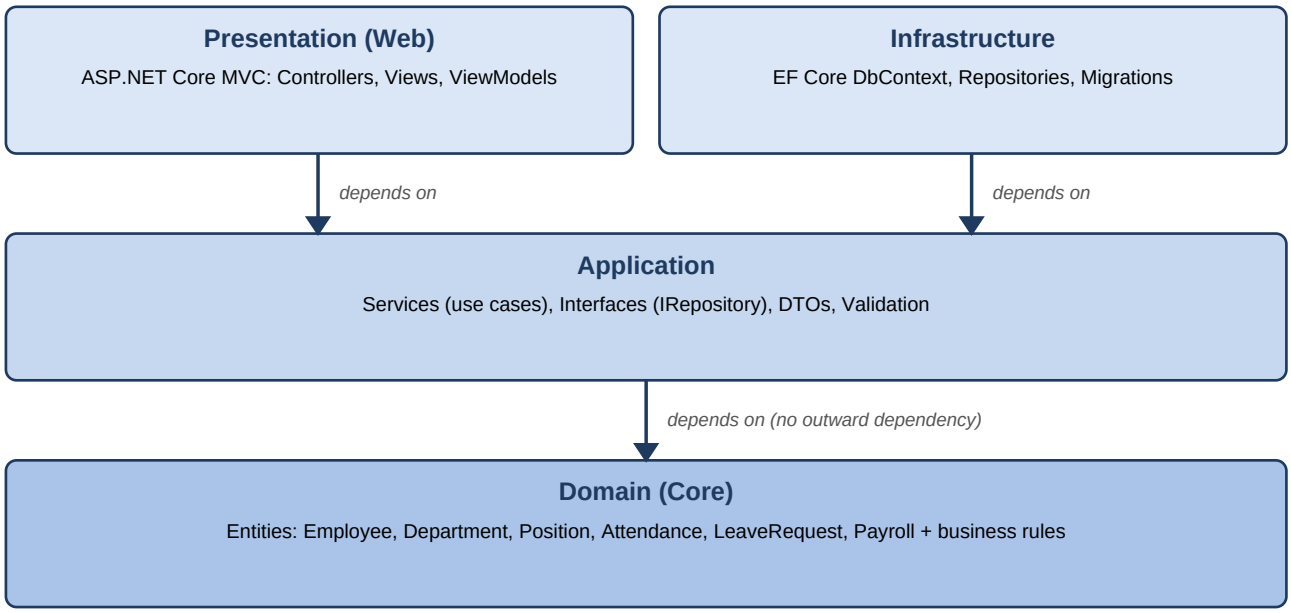
6.2 Step 2 - N-tier, Clean (Onion) and Layered

Pattern	Idea	Pros	Cons
N-tier (physical)	Splits the system into tiers deployed separately (client, web server, application server, database).	Independent deployment and scaling.	Network overhead; does not by itself organize the code.
Layered	Code is split into horizontal layers (UI, Business, Data); each layer depends on the one below.	Simple, familiar, quick to build.	Business layer depends on the data layer; harder to test and replace the database.
Clean / Onion	A layered style where dependencies point inward to the Domain at the center; outer layers depend on inner ones through interfaces.	Business logic independent of UI, database and frameworks; very testable and maintainable.	More projects, folders and interfaces; a bit more initial work.

Clean (Onion) architecture is a **layered architecture** with a stricter dependency rule, so it keeps the benefits of layering and fixes its main weakness.

6.3 Step 3 - Select Clean Architecture (for the EMS example)

Decision: Clean (Onion) Architecture. It matches the criteria: payroll and leave rules live in the Domain/Application layers and can be tested without a database; the UI or database can change without touching business rules; and new modules can be added in the same structure.



6.4 Step 4 - Project: ASP.NET MVC with the Chosen Architecture

One Visual Studio solution with four projects, following the dependency direction (Web and Infrastructure depend on Application; Application depends on Domain):

```
EMS.sln
|-- EMS.Domain            (Class Library)   no dependencies
|     |-- Entities/        Employee.cs, Department.cs, Position.cs,
|     |                    Attendance.cs, LeaveRequest.cs, Payroll.cs
|     |-- Enums/           LeaveStatus.cs
|-- EMS.Application       (Class Library)   -> Domain
|     |-- Interfaces/      IEmployeeRepository.cs, ILeaveRepository.cs ...
|     |-- Services/        EmployeeService.cs, LeaveService.cs, PayrollService.cs
|     |-- DTOs/            EmployeeDto.cs, CreateEmployeeDto.cs
|-- EMS.Infrastructure    (Class Library)   -> Application, Domain
|     |-- Data/            AppDbContext.cs, Migrations/
|     |-- Repositories/    EmployeeRepository.cs, LeaveRepository.cs ...
|-- EMS.Web               (ASP.NET Core MVC) -> Application, Infrastructure
      |-- Controllers/     EmployeesController.cs, LeavesController.cs ...
      |-- Models/          EmployeeViewModel.cs
      |-- Views/           Employees/Index.cshtml, Create.cshtml ...
      |-- Program.cs       Dependency Injection setup
```

6.5 Step 5 - Allocate Classes in Specs Document

Every class from the specs document is assigned to a layer and a folder:

Class / Item	Layer (Project)	Folder
Employee, Department, Position, Attendance, LeaveRequest, Payroll	Domain (EMS.Domain)	Entities/
LeaveStatus (enum)	Domain (EMS.Domain)	Enums/
IEmployeeRepository, ILeaveRepository, IPayrollRepository	Application (EMS.Application)	Interfaces/
EmployeeService, LeaveService, PayrollService	Application (EMS.Application)	Services/
EmployeeDto, CreateEmployeeDto	Application (EMS.Application)	DTOs/
AppDbContext	Infrastructure (EMS.Infrastructure)	Data/
EmployeeRepository, LeaveRepository, PayrollRepository	Infrastructure (EMS.Infrastructure)	Repositories/

Class / Item	Layer (Project)	Folder
EmployeesController, LeavesController, AttendanceController	Presentation (EMS.Web)	Controllers/
EmployeeViewModel and the .cshtml views	Presentation (EMS.Web)	Models/ and Views/

References

- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Palermo, J. (2008). *The Onion Architecture*. https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/
- Microsoft Learn. *Common web application architectures* (ASP.NET Core). https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures
- Sommerville, I. *Software Engineering*. Pearson. (Requirements engineering and architectural design chapters).
- IEEE Std 830-1998 / ISO/IEC/IEEE 29148: *Software Requirements Specifications*.
- Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley.